Plan for Kolte Patil Project Portal V.2

1. Overview

This project introduces a centralized system for managing and distributing project listings across:

- **Admin Web Portal** (for backend project management)
- **Mobile Application** (existing consumer of project data)

The backend will act as the single source of truth, exposing a unified API consumed exclusively by the mobile platform. *(Note: The User Web Portal has been removed from scope).*

2. Objective

- Enable admins to securely create, update, hide, and archive projects
- Ensure projects are accurately visible on the mobile app
- Maintain a strict API contract for mobile app compatibility
- Ensure real-time data consistency

3. User Stories (Summary)

	As a...	I want to...	So that...
1	Admin	create a new project	it can be listed for users on the mobile app
2	Admin	add and manage project details	users can understand details about the project
3	Admin	upload a project brochure & images	users can view visually represented information
4	Admin	view all created projects	I can manage existing listings
5	Admin	edit an existing project	I can keep project information up to date

6	Admin	toggle active/inactive status	I can temporarily hide a project without deleting it
7	Admin	Delete a project	It is permanently deleted from the db
8	Admin	archive a project	it is not permanently removed from the active mobile view
9	App User	view a list of projects & details	I can explore and evaluate available options
10	App User	navigate paginated project listings	I can browse efficiently

4. User Stories & Functional Scope (Detailed)

4.1 Create Project (Admin)

- **Scope**: Input project details explicitly mapped to the Mobile App API response. The form will contain:
  - `Project Name` (Text string)
  - `Description` (Text string)
  - `Location` (Text string)
  - `Bedrooms` (Numeric)
  - `Furnishing` (Dropdown / Text)
  - `Floor` (Text string)
  - `Preference` (Text string)
  - `Area` (Text string)
  - `Project Brochure` (File Upload → generates AWS S3 URL)
  - `Attachments` (Multiple Image Uploads → generates array of AWS S3 URLs)
- **Outcome**: Project appears via the API exactly matching the legacy format.

4.2 View, Update, Toggle, and Archive Projects (Admin)

- **Scope**: Admin can view paginated lists of projects, modify details, hide a project temporarily (`is_active = false`), or soft-delete it completely (`is_archived = true`).

- **Outcome**: Changes immediately reflect in the API; archived or inactive projects are excluded.

4.3 Mobile App Data Consumption & API Integrity

- **Scope**: Consume `/projects/list/` preserving the exact response structure.
- **Constraint**: *No changes required in the existing mobile app.*

---

5. System Architecture & Components

[System Architecture Diagram]

System Architecture Diagram

1. **Admin Web Portal**: Secure application for authorized staff.
2. **Mobile Application**: Existing consumer of the unified API.
3. **Backend API Service**: Node.js/Express service providing distinct secure vs public endpoints.
4. **Database**: **PostgreSQL Database** storing project and admin data.
5. **Media Storage**: **AWS S3** for native handling and hosting of uploaded brochures and images.

---

6. Admin Authentication Flow

1. **Login Event**: Admin submits credentials to `POST /admin/auth/login`.
2. **Verification**: Backend verifies the hashed password against the `Admins` table.
3. **Session Issuance & Access**: Backend issues a JSON Web Token (JWT), passed via `Authorization: Bearer <Token>`.

> [!NOTE] **Authentication Scope:** As requested, authentication will be limited to a basic, robust secured JWT login. More advanced security hardening strategies (complex RBAC, IP whitelisting) will be deferred to the subsequent VAPT (Vulnerability Assessment and Penetration Testing) phase.

---

7. Database Schema Design (PostgreSQL)

Table: `Projects`

- `id` (PK, Auto-increment) → Maps to `projectId`
- `project_name` (VARCHAR), `description` (TEXT), `location` (VARCHAR)
- **Overview Fields**: `bedrooms` (INT), `furnishing` (VARCHAR), `floor` (VARCHAR), `preference` (VARCHAR), `area` (VARCHAR)

- **Media**: `project_brochure` (VARCHAR URL)
- **State**: `is_active` (BOOLEAN, default: true), `is_archived` (BOOLEAN, default: false)
- **Audit Logging**: `created_by` (INT FK), `updated_by` (INT FK)
- **Analytics**: `click_count` (INT, default: 0)
- `created_at`, `updated_at` (TIMESTAMP)

Table: `Project_Attachments`

- `id` (PK)
- `project_id` (FK References `Projects(id)`)
- `name` (VARCHAR)
- `image_url` (VARCHAR URL)
- `extension` (VARCHAR, e.g., 'PNG', 'JPG')

Table: `Admins`

- `id` (PK)
- `email` (VARCHAR Unique), `password_hash` (VARCHAR)
- `created_at`, `updated_at` (TIMESTAMP)

---

8. Complete API Overview

Endpoint	Method	Purpose	Constraint
`/projects/list/`	POST	Fetch paginated active project list	Must match legacy format
`/projects/:id/click`	POST	Increment analytics click count	Tracks popularity
`/admin/auth/login`	POST	Authenticate portal admins	Requires secure credential
`/admin/auth/logout`	POST	Invalidate Token	Secure Logout
`/admin/projects/`	GET	Fetch paginated catalog for Admins	Returns all data

`/admin/projects/`	POST	Create a new project record	Requires Auth Bearer Token
`/admin/projects/:id`	PUT	Edit project details	Requires Auth Bearer Token
`/admin/projects/:id`	PATCH	Toggle Active/Inactive	Requires Auth Bearer Token
`/admin/projects/:id`	DELETE	Soft delete (Archive) a project	Requires Auth Bearer Token

Exact Mobile API Contract (`POST /projects/list/`)

Headers Required:

- `Access-Token` : `<token>`

Success Response (200 OK):

```
{
  "status_code": 200,
  "status_message": "Success",
  "response_data": [
    {
      "projectId": 12356,
      "projectName": "Casa Venero",
      "description": "test",
      "location": "dubai",
      "overview": {
        "bedrooms": 3,
        "furnishing": "Furnished",
        "floor": "2nd Floor",
        "preference": "Vegetarian",
        "area": "2000 sqft"
      },
      "project_brochure": "<https://s3.amazonaws.com/brochure.mp4>",
      "attachments": [
        {
          "name": "img_1",
          "imageUrl": "<https://s3.amazonaws.com/image.jpg>",
          "extension": "JPG"
        }
      ]
    }
  ],
  "pagination": {
    "current_page": 1,
    "per_page": 10,
    "total_items": 627,
    "total_pages": 63,
    "has_next": true,
    "has_prev": false
  }
}
```

```
36
```

---

9. File Upload & Media Handling

As explicitly required, full media upload functionality is restored to the architecture:

- When admins create a project, they will upload the brochure and image attachments directly to an AWS S3 bucket.
- The S3 URLs will be saved persistently onto PostgreSQL and distributed natively out to the mobile app via the API response payload.

---

10. Technical Data Flows

Flow A: Admin Creates a Project

Flow A Diagram

Flow A Diagram

Flow B: App Consumes the Unified API

Flow B Diagram

Flow B Diagram

---

11. Caching & Performance

Given Mobile app traffic, read-replica or Redis caching strategies will be discussed later during load testing, to prevent 600+ project payloads from bottlenecking PostgreSQL.

---

12. Deployment, Concurrency, and Deletion Policies

- **Concurrency Rate Risk**: Only one admin will be making changes at a time, so this is fully taken care of.
- **Archive Strategy**: `is_archived = true` and `is_active = false` serve as non-destructive state filters so administrators never permanently lose data.

---

13. Metrics & Analytics Tracking

1. **Detailed View Clicks**: Every time a user explicitly opens a project's detailed view, the client fires a request to `POST /projects/:id/click`.

2. **Data Storage**: The backend increments the `click_count` integer field on the `Projects` table.

---

14. Timeline

- **Backend API & PostgreSQL Configuration**: 2–3 days
- **Admin Portal Frontend**: 2–3 days
- **Testing, Q/A, Validation**: 2 days
- **Total Estimated Timeline**: ~6–8 days

---

15. Success Criteria

- ✅ Admin can manage projects, upload S3 media, entirely securely via complete endpoints.
- ✅ Projects are instantaneously visible on the Mobile App based on `is_active` toggles.
- ✅ API response aligns with the payload contract with absolutely **no deviations**.